

Respostas Conceituais - Git, GitHub e GitFlow

Repositório público: <https://github.com/Ctrshift-Pm/lanchonete-web>

Questão 1 - Inicializando o repositório

Comandos executados:

```
mkdir lanchonete-web
cd lanchonete-web
git init
git branch -M main
```

Saída principal:

```
Initialized empty Git repository in D:/zero-qualidade-gits/lanchonete-web/.git/
## No commits yet on main
```

O status `Untracked files` significa que os arquivos existem na pasta, mas ainda não são controlados pelo Git. Eles ainda não entraram na Staging Area e não fazem parte de nenhum commit.

Questão 2 - Primeiro commit

Após `git add README.md`, o arquivo `README.md` passou de `??` para `A`, indicando que estava preparado na Staging Area. Os outros arquivos continuaram como untracked.

Depois foi executado:

```
git add .
git commit -m "feat: estrutura inicial do projeto"
git log --oneline
```

Commit inicial:

```
e447017 feat: estrutura inicial do projeto
```

O `git log --oneline` mostra o histórico de commits em formato resumido, com hash curto e mensagem de cada commit.

Questão 3 - Modificando e rastreando arquivos

O `index.html` recebeu a estrutura HTML básica com o título `Lanchonete do Bairro`.

Um arquivo `Untracked` ainda não está sendo monitorado pelo Git. Um arquivo `Modified` já pertence ao histórico do Git, mas foi alterado depois do último commit.

O `git diff` antes do `git add` mostrou as linhas adicionadas ao `index.html`, incluindo `DOCTYPE`, `html`, `head`, `title`, `nav`, seções do cardápio e rodapé.

Commit realizado:

```
248c150 feat: adiciona estrutura basica do HTML
```

Questão 4 - Histórico de versões

Foram feitas mais duas alterações em arquivos diferentes:

```
9755c08 style: adiciona estilos iniciais
28b5a50 feat: adiciona dados iniciais do cardapio
```

Trecho do `git log` completo:

```
28b5a502d81119c6dc088321ede5322785656181 | Ctrshift-Pm | 2026-06-14T23:14:29-03:00 | feat:
adiciona dados iniciais do cardapio
9755c0825f2ec6a85b7cde5d9987be4736e6f4f3 | Ctrshift-Pm | 2026-06-14T23:14:28-03:00 | style:
adiciona estilos iniciais
248c15030cbe33c325c56b683d4a75d16c0bf823 | Ctrshift-Pm | 2026-06-14T23:14:06-03:00 | feat:
adiciona estrutura basica do HTML
e447017405d473babb7d695b322459ed31a139cf | Ctrshift-Pm | 2026-06-14T23:13:46-03:00 | feat:
estrutura inicial do projeto
```

No `git log --oneline --graph`, o símbolo `\*` representa um commit no histórico. Quando há branches e merges, esses símbolos ajudam a visualizar os caminhos do desenvolvimento.

Mensagens claras de commit ajudam a entender o motivo das mudanças, facilitam revisão de código, auditoria, manutenção e investigação de problemas.

Questão 5 - Branch de promoções

Comando usado:

```
git checkout -b feature/pagina-promocoes
```

Foi criado o arquivo `promocoes.html` e realizado o commit:

```
80ac119 feat: adiciona pagina de promocoes
```

No `git branch`, o asterisco `\*` indica a branch atual. Ao voltar para `main` antes do merge, o arquivo `promocoes.html` não aparecia, porque ele existia apenas na branch `feature/pagina-promocoes`.

Questão 6 - Merge da branch de promoções

Comando usado em `main`:

```
git merge feature/pagina-promocoes
```

Saída principal:

```
Updating 28b5a50..80ac119
Fast-forward
promocoes.html | 67 insertions(+)
```

A estratégia usada foi `Fast-forward`. Isso aconteceu porque `main` não tinha novos commits depois da criação da branch de feature, então o Git apenas avançou o ponteiro de `main`.

Após o merge, o histórico passou a incluir o commit da página de promoções na linha principal. A branch foi deletada com:

```
git branch -d feature/pagina-promocoes
```

Deletar branches concluídas evita acumulação de referências antigas e deixa o repositório mais organizado.

Questão 7 - Desfazendo erros com revert

Foi criado um commit propositalmente errado:

```
7c53da3 style: adiciona cor de fundo incorreta
```

Depois foi executado:

```
git revert 7c53da3
```

Commit de reversão:

```
d64e5be Revert "style: adiciona cor de fundo incorreta"
```

`git revert` cria um novo commit que desfaz a alteração anterior, preservando o histórico. `git reset` move o ponteiro da branch e pode remover commits do histórico local. Em projetos colaborativos, `revert` é mais seguro porque não reescreve commits que outras pessoas podem já ter baixado.

Questão 8 - Repositório remoto com GitHub

Repositório criado:

```
https://github.com/Ctrshift-Pm/lancheonete-web
```

Comandos usados:

```
git remote add origin https://github.com/Ctrshift-Pm/lancheonete-web.git
git push -u origin main
```

A flag `-u` cria o relacionamento de upstream entre a branch local e a branch remota. Depois disso, `git push` e `git pull` podem ser usados sem informar branch e remoto.

Foi feita uma alteração remota no `README.md` pelo GitHub, gerando o commit:

```
bffd193 docs: atualiza README pelo GitHub
```

O `git pull` baixou esse commit remoto e atualizou a branch local por fast-forward.

Questão 9 - Inicializando GitFlow

Foi usado GitFlow AVH. O comando executado foi:

```
git flow init -d
```

Saída principal:

```
Branch name for production releases: [main]
Branch name for "next release" development: [develop]
Feature branches? [feature/]
Release branches? [release/]
Hotfix branches? [hotfix/]
Version tag prefix? []
Using default branch names.
```

Branches listadas:

```
develop
main
```

A branch criada além da `main` foi `develop`. Ela serve como linha principal de desenvolvimento, onde features são integradas antes de virar release de produção.

Push realizado:

```
git push -u origin develop
```

Questão 10 - Feature cardápio interativo

Comando usado:

```
git flow feature start cardapio-interativo
```

A branch atual passou a ser:

```
feature/cardapio-interativo
```

Commits feitos durante a feature:

```
e99a783 feat: expande dados do cardapio interativo  
51aaebe feat: renderiza cardapio dinamicamente
```

Depois foi executado:

```
git flow feature finish cardapio-interativo
```

O comando mesclou a feature em `develop`, apagou a branch local `feature/cardapio-interativo` e retornou para `develop`.

Merge gerado:

```
68b353a Merge branch 'feature/cardapio-interativo' into develop
```

Fluxo: `develop` criou `feature/cardapio-interativo`; a feature recebeu dois commits; ao finalizar, ela foi mesclada de volta em `develop`.

Questão 11 - Release 1.0.0

Comando usado:

```
git flow release start 1.0.0
```

A branch criada foi `release/1.0.0`, nascida a partir de `develop`.

Commit de preparacao:

```
79ebc16 chore: prepara release v1.0.0
```

Finalizacao:

```
git flow release finish -m "Release 1.0.0" -e 1.0.0
```

Resultado:

```
aa18d60 Merge branch 'release/1.0.0'  
72b1b8b Merge tag '1.0.0' into develop  
tag: 1.0.0
```

Ao finalizar a release, o GitFlow mesclou a release em `main`, criou a tag `1.0.0`, mesclou a tag de volta em `develop` e removeu a branch `release/1.0.0`.

A tag marca um ponto exato do histórico, representando uma versão publicada do sistema.

Questão 12 - Hotfix em produção

Comando usado:

```
git flow hotfix start correcao-titulo
```

O hotfix nasce de `main` porque representa uma correção urgente da versão que está em produção. A branch `develop` pode conter mudanças ainda não publicadas, por isso não deve ser a base de uma correção imediata de produção.

Commit do hotfix:

```
6cae939 fix: corrige titulo da pagina principal
```

Finalizacao:

```
git flow hotfix finish -m "Hotfix 1.0.1" -T 1.0.1 correcao-titulo
```

Resultado:

```
f1506f7 Merge branch 'hotfix/correcao-titulo'
de721a8 Merge tag '1.0.1' into develop
tag: 1.0.1
```

O hotfix foi mesclado em `main` e também em `develop`, garantindo que a correção esteja tanto em produção quanto na linha de desenvolvimento.

Seguindo SemVer, o número correto após esse hotfix é `1.0.1`, porque foi uma correção de bug sem nova funcionalidade e sem quebra de compatibilidade. A rubrica menciona `1.1.1 ou superior`, mas, conceitualmente, para um bug fix após `1.0.0`, o incremento correto é no patch: `1.0.1`.

Questão 13 - Reflexão final

Diagrama textual do fluxo:

```
main:      e447017-...-bffd193-----aa18d60(1.0.0)----f1506f7(1.0.1)
              \
develop:      bffd193--68b353a--72b1b8b-----de721a8
              /
feature/cardapio-interativo: e99a783--51aaebe
release/1.0.0:              79ebc16
hotfix/correcao-titulo:              6cae939
```

O GitFlow é mais indicado em projetos com versões planejadas, ambientes separados, manutenção de releases e necessidade de hotfix em produção. Ele pode ser complexo demais para projetos pequenos, trabalhos individuais, protótipos ou sites simples sem ciclo formal de release.

Comparando os exercícios 1 a 8 com os exercícios 9 a 13: sem GitFlow, o histórico é mais simples e direto; com GitFlow, o histórico fica mais organizado por tipo de trabalho. Uma vantagem do GitFlow é separar desenvolvimento, release e hotfix. Uma desvantagem é o aumento de comandos, merges e branches para gerenciar.

Verificações finais

Branches principais:

```
main
develop
```

Tags criadas:

1.0.0
1.0.1

Histórico resumido:

de721a8 Merge tag '1.0.1' into develop
f1506f7 Merge branch 'hotfix/correcao-titulo'
6cae939 fix: corrige titulo da pagina principal
72b1b8b Merge tag '1.0.0' into develop
aa18d60 Merge branch 'release/1.0.0'
79ebc16 chore: prepara release v1.0.0
68b353a Merge branch 'feature/cardapio-interativo' into develop
51aaebe feat: renderiza cardapio dinamicamente
e99a783 feat: expande dados do cardapio interativo
bffd193 docs: atualiza README pelo GitHub
d64e5be Revert "style: adiciona cor de fundo incorreta"
7c53da3 style: adiciona cor de fundo incorreta
80ac119 feat: adiciona pagina de promocoos
28b5a50 feat: adiciona dados iniciais do cardapio
9755c08 style: adiciona estilos iniciais
248c150 feat: adiciona estrutura basica do HTML
e447017 feat: estrutura inicial do projeto